GREENN SCHOOL · MÓDULO LARAVEL

L3

Laravel: rotas e views com Blade

Apostila do aluno · duração da aula: 2h30

O que você vai aprender

1. Rotas a fundo: parâmetros e `where`
2. Rotas nomeadas, grupos, redirect e 404
3. Blade: `{{ }}`, condicionais e laços
4. Layouts, partials e componentes
5. O projeto da aula: o site da escola

Antes de começar

Na L2 você viu o caminho de todo pedido no Laravel: **rota → controller → view**. Esta aula aprofunda as duas pontas desse caminho. Do lado das **rotas**: URLs com pedaços que mudam (`/cursos/3`), nomes para não escrever URL à mão e respostas de erro. Do lado das **views**: o **Blade**, o jeito do Laravel escrever HTML com dados, que troca o `<?= ?>` por `{{ }}` e evita copiar o mesmo cabeçalho em todas as páginas. No site **site-alunos-nu.vercel.app** (módulo Laravel), o playground desta aula traz um mini-Laravel com Blade para você testar tudo sem instalar nada.

Prof. Walter Caciano · Greenn School

1 Rotas a fundo: parâmetros e where

Todas as URLs do site ficam em `routes/web.php`. Uma rota pode responder de três jeitos: com uma **função** (bom para testar rápido), com um **controller** (o jeito do dia a dia) ou com `Route::view`, quando a página só mostra uma view e não tem lógica nenhuma.

routes/web.php

```php
<?php
use App\Http\Controllers\CursoController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {                          // ① uma função
    return 'Bem-vindo à Greenn School!';
});

Route::get('/cursos', [CursoController::class, 'index']);   // ② um controller

Route::view('/sobre', 'sobre', ['escola' => 'Greenn']);    // ③ só uma view
```

Um pedaço da URL entre chaves é um **parâmetro**: ele chega na função (ou no método do controller) na ordem em que aparece. Com `?` no fim ele é **opcional**, e o parâmetro da função precisa de um valor padrão. Para aceitar só certos valores, use `whereNumber` ou `where` com uma expressão regular.

```php
Route::get('/cursos/{id}', function (string $id) {
    return "Curso número $id";
})->whereNumber('id');                                 // /cursos/abc → 404

Route::get('/cursos/{curso}/aulas/{aula}', function (string $curso, string $aula) {
    return "Curso $curso, aula $aula";                 // /cursos/2/aulas/5
});

Route::get('/ola/{nome?}', function (string $nome = 'visitante') {
    return "Olá, $nome!";                              // /ola → Olá, visitante!
});

Route::get('/blog/{slug}', fn (string $slug) => "Post: $slug")
    ->where('slug', '[a-z-]+');                        // só minúsculas e hífen
```

> `fn () => ...` é a forma curta de `function () { return ...; }`, como a arrow function do JavaScript. Ela devolve o valor da expressão sem precisar de `return`.

2 Rotas nomeadas, grupos, redirect e 404

Se você escreve `/cursos/3` em 20 links e um dia a URL muda, tem que caçar os 20. Por isso a gente dá um **nome** à rota e monta os links com `route()`: mudou a URL, os links acompanham.

O costume é `assunto.ação`.

```
Route::get('/cursos', [CursoController::class, 'index'])->name('cursos.index');
Route::get('/cursos/{id}', [CursoController::class, 'show'])->name('cursos.show');

route('cursos.index');            // http://localhost:8000/cursos
route('cursos.show', 3);          // http://localhost:8000/cursos/3
route('cursos.show', ['id' => 3]); // o mesmo, dizendo o nome do parâmetro
```

Mais três ferramentas do dia a dia: **`abort(404)`** para tudo e responde com erro quando o que foi pedido não existe (ou `abort(403)`, acesso proibido); **`Route::redirect`** manda uma URL antiga para a nova; e o **grupo** coloca o mesmo prefixo na URL e no nome de várias rotas, sem repetir.

```
Route::redirect('/formacoes', '/cursos');             // resposta 302

Route::prefix('admin')->name('admin.')->group(function () {
    Route::get('/painel', [PainelController::class, 'index'])->name('painel');
    Route::get('/alunos', [AlunoController::class, 'index'])->name('alunos');
});
// URLs: /admin/painel e /admin/alunos · nomes: admin.painel e admin.alunos

public function show(string $id)                      // no controller
{
    $curso = Curso::find($id);
    if (!$curso) {
        abort(404);                                    // 404 | Not Found
    }
    return view('cursos.show', ['curso' => $curso]);
}
```

> **Para ver todas as rotas** do projeto, com URL, nome e controller: `php artisan route:list`. É o primeiro lugar para olhar quando aparece um 404 ou o erro *Route [cursos.show] not defined* (o nome não existe).

3 Blade: {{ }}, condicionais e laços

As views do Laravel ficam em `resources/views` e terminam em `.blade.php`. O **Blade** é o jeito do Laravel escrever HTML com dados: `{{ $nome }}` faz o papel do `<?= $nome ?>` da L2, e as estruturas de controle viram **diretivas** que começam com `@`. A view `cursos.index` é o arquivo `resources/views/cursos/index.blade.php`: o ponto separa as pastas.

PHP puro (L2)	Blade (L3)	Observação
`<?= $nome ?>`	`{{ $nome }}`	escapa o HTML: protege contra XSS
`<?= $html ?>`	`{!! $html !!}`	sem escapar: só HTML em que você confia
`<?php if ($x): ?>`	`@if ($x)`	`@elseif`, `@else`, `@endif`
`<?php if (!$x): ?>`	`@unless ($x)`	um "se não"; fecha com `@endunless`
`<?php foreach ($a as $i): ?>`	`@foreach ($a as $i)`	fecha com `@endforeach`
(um if em volta do foreach)	`@forelse … @empty`	foreach com plano B para lista vazia
`<?php /* … */ ?>`	`{{-- … --}}`	comentário: não vai para o HTML

Por que escapar? Se um aluno cadastra o nome `<script>alert("hackeado")</script>` e a página mostra esse nome sem cuidado, o código roda no navegador de quem abrir a página. Esse ataque se chama **XSS**. O `{{ }}` troca `<` por `<` e o código vira texto inofensivo. Regra: tudo o que vem do usuário vai em `{{ }}`.

resources/views/boletim.blade.php

```
<h1>Boletim de {{ $aluno }}</h1>

@if ($media >= 7)
  <p>Aprovado(a) com média {{ $media }}</p>
@elseif ($media >= 5)
  <p>Recuperação</p>
@else
  <p>Reprovado(a)</p>
@endif

<ul>
@forelse ($cursos as $curso)
  <li>{{ $loop->iteration }}. {{ $curso }}</li>
@empty
  <li>Nenhum curso ainda.</li>
@endforelse
</ul>
```

Dentro de todo `@foreach` (e `@forelse`) existe a variável `$loop`, que diz em que volta você está:

Propriedade	O que é	Na 1ª volta de 4
`$loop->iteration`	a volta atual, começando em 1	1
`$loop->index`	a volta atual, começando em 0	0
`$loop->first` / `$loop->last`	true na primeira / na última volta	true / false
`$loop->count`	quantos itens a lista tem	4
`$loop->remaining`	quantas voltas faltam	3
`$loop->even` / `$loop->odd`	volta par / ímpar	false / true

4 Layouts, partials e componentes

Todas as páginas de um site têm o mesmo cabeçalho, menu e rodapé. Em vez de copiar, você escreve **um layout** com buracos marcados por `@yield`. Cada página diz qual layout usa com `@extends` e preenche os buracos com `@section`.

home.blade.php

```
@extends('layouts.app')
@section('conteudo')
  <h1>Bem-vindo</h1>
@endsection
```

preenche →

layouts/app.blade.php

```
<header>…</header>
@yield('conteudo')
<footer>…</footer>
```

=

página pronta

```
<header>…</header>
<h1>Bem-vindo</h1>
<footer>…</footer>
```

resources/views/layouts/app.blade.php

```blade
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <title>@yield('titulo', 'Greenn School') | Greenn School</title>
</head>
<body>
  @include('partials.menu')
  <main>
    @yield('conteudo')
  </main>
  <footer>Aulas aos sábados</footer>
</body>
</html>
```

resources/views/home.blade.php

```
@extends('layouts.app')

@section('titulo', 'Início')                          {{-- forma de uma linha --}}

@section('conteudo')
  <h1>Bem-vindo, {{ $nome }}!</h1>
  @foreach ($cursos as $c)
    @include('partials.card-curso', ['curso' => $c])
  @endforeach
@endsection
```

O **partial** é um pedaço reaproveitado (o menu, o card de um curso), colocado com `@include`. Ele enxerga as variáveis da view que o incluiu, e você pode mandar mais dados no segundo argumento. Por costume, os partials ficam em `resources/views/partials`.

O **componente** é um partial que se usa como tag HTML: o arquivo `resources/views/components/alerta.blade.php` vira `<x-alerta>`. O que vai entre as tags chega em `$slot`; os atributos viram variáveis, e o `@props` define os valores padrão. Com dois-pontos (`:tipo="$nivel"`), o valor é uma expressão PHP. Os componentes voltam com força nos formulários (L5).

resources/views/components/alerta.blade.php

```
@props(['tipo' => 'info'])
<div class="alerta {{ $tipo }}">{{ $slot }}</div>

{{-- usando, em qualquer view: --}}
<x-alerta tipo="erro">Falta pagar a mensalidade.</x-alerta>
<x-alerta>Aula sábado às 9h.</x-alerta>
```

5 O projeto da aula: o site da escola

Juntando tudo: uma lista de cursos em `/cursos`, a página de cada curso em `/cursos/{id}` (ou 404) e as duas usando o mesmo layout, com links feitos por `route()`. Por enquanto o model devolve uma lista fixa; na L4 os cursos vêm do banco de dados.

routes/web.php

```
Route::get('/cursos', [CursoController::class, 'index'])->name('cursos.index');
Route::get('/cursos/{id}', [CursoController::class, 'show'])
    ->whereNumber('id')->name('cursos.show');
```

app/Http/Controllers/CursoController.php

```php
<?php
namespace App\Http\Controllers;

use App\Models\Curso;

class CursoController extends Controller
{
    public function index()
    {
        return view('cursos.index', ['cursos' => Curso::all()]);
    }

    public function show(string $id)
    {
        $curso = Curso::find($id);
        if (!$curso) {
            abort(404);
        }
        return view('cursos.show', ['curso' => $curso]);
    }
}
```

resources/views/cursos/index.blade.php

```blade
@extends('layouts.app')

@section('conteudo')
  <h1>Nossos cursos</h1>
  <ul>
  @foreach ($cursos as $curso)
    <li><a href="{{ route('cursos.show', $curso->id) }}">{{ $curso->nome }}</a></li>
  @endforeach
  </ul>
@endsection
```

No seu computador, num projeto Laravel: o controller é criado com `php artisan make:controller CursoController`; as views são arquivos novos em `resources/views` (crie a pasta `cursos` e o arquivo `index.blade.php`); e `php artisan serve` abre o site em `http://localhost:8000`. O Blade é compilado para PHP comum e guardado em `storage/framework/views`: se uma view parecer não atualizar, rode `php artisan view:clear`.

6 Exercícios

Do 1 ao 3, rotas e o escape do Blade; 4 a 7, condicionais, laços e o `$loop`; 8 a 10, layout, partials e o caminho completo. Em cada um, compare a sua saída com a esperada: cada letra, espaço e acento conta.

01 Rotas com parâmetro NÍVEL 1

Rotas `/alunos/{nome}` e `/dobro/{n}`, esta só com números. · implementação

Saída esperada: Perfil de Ana / O dobro de 21 é 42 / 404 | Página não encontrada

02 Parâmetro opcional e rota nomeada NÍVEL 1

`/ola/{nome?}` com valor padrão, nome `ola` e `route()`. · implementação

Saída esperada: Olá, visitante! / Olá, Bia! / Link: /ola/Bia

03 {{ }} ou {!! !!}? NÍVEL 1

Uma view que escapa o que não devia e não escapa o que devia. · previsão de saída e correção

Saída esperada: o nome escapado e o destaque como HTML

04 @if no boletim NÍVEL 2

Uma view com `@if` / `@elseif` / `@else` para a situação do aluno. · implementação

Saída esperada: Ana: aprovado(a) / Caio: recuperação / Bia: reprovado(a)

05 @forelse da turma NÍVEL 2

Lista de alunos com `@forelse` e o plano B do `@empty`. · implementação

Saída esperada: dois <li> na manhã; “Nenhum aluno matriculado” na noite

06 Ranking com $loop NÍVEL 2

Ranking numerado com `$loop->iteration`, `first`, `last` e `count`. · implementação

Saída esperada: 1º Ana / 🏆 Primeiro lugar! / 2º Caio / 3º Bia / Fim do ranking: 3 alunos

07 Conserte a view NÍVEL 2

Uma view com três erros de Blade para encontrar. · identificação de erros

Saída esperada: Ana: 9.5 ✅ / Caio: 6

08 Página com layout NÍVEL 2

View `contato` com `@extends` e duas `@section`. · implementação

Saída esperada: a página de contato dentro do layout (título, cabeçalho e rodapé)

09 Refatore com @include NÍVEL 3

Tirar o card repetido para `partials.produto` e usar em duas views. · refatoração

Saída esperada: a loja e as promoções com o mesmo card

10 Desafio: catálogo de cursos DESAFIO

Rota `/cursos` → `CursoController` → views com layout, links por `route()` e 404. · integração

Saída esperada: HTML renderizado na prévia e 404 para um curso que não existe

Travou? Volte ao exemplo da seção correspondente, rode o código no site e mude uma coisa por vez para entender o que cada parte faz. Erro dentro de uma view? Confira se todo `@if` tem `@endif`, todo `@foreach` tem `@endforeach` e se as variáveis têm `$`.